

Activity Is Not Progress: Which Runtime Signals Reveal That a Coding Agent Has Stalled?

Qichen Tong Ke Chuang Jiang

September 11, 2026

Abstract

Coding agents spend much of their runtime on activity that does not advance the task. We ask which runtime signals let an online, reference-free monitor recognise this, and answer with annotated evidence rather than a new model. Progress is defined through three channels — epistemic (new task-relevant information), implementation (durable plausible change) and verification (objective evidence about correctness) — and 1198 windows over 81 trajectories are labelled with a written codebook, then re-read under reduced context to measure stability (88.1% binary agreement, $\kappa = 0.70$). On 1500 Terminal-Bench 2.0 trajectories (45 tasks, 9 scaffolds) under task-level cross-validation we compare six signal families: action repetition, relevance-blind novelty, task-grounded evidence, verification deltas, workspace churn, and rolling semantic redundancy of successive steps.

Two findings. First, the family we expected to dominate does not: task-grounded evidence scores 0.587, below exact repetition (0.608) and far below semantic redundancy (0.775; paired Δ -0.193, clustered 95% interval excluding zero). Verification deltas and workspace churn are weaker still. Second, and contrary to our earlier reading, the label is an *episode* rather than a point: consecutive windows sharing a label merge into 68 contiguous stall episodes with a median duration of 16 steps. Scored at that unit the picture reverses. Inside a single run, and against productive stretches at matched run positions, the monitor separates 30 of 36 stall episodes (83%, clustered interval $[+0.074, +0.301]$, $p = 0.0005$), while a run-position proxy separates only 12 of 36.

That separation is not by itself an alarm, because the score drifts upward with run progress. The drift is what the detector removes: smooth the score over 3 windows, anchor a reference to the run’s own first 15% of steps — before a stall can have begun — and alarm when the smoothed score exceeds $\mu + 1.0\sigma$. The resulting alarm flags 53% of stall episodes at a 17% per-window false-alarm rate with median latency zero steps, and reaches 56% at 18% under a nested protocol that selects every parameter on held-out runs. It uses no labels, no reference trajectories and no task-specific tuning, and we report both where it succeeds and where it fails: roughly two episodes in five are missed, and short stalls are the ones missed. Everything needed to reproduce this is released, including the annotated windows with step-cited justifications, the frozen feature matrices, the monitor implementation (0.33 ms per feature vector on one CPU core), and the scripts that regenerate every figure and every number in this paper.

Keywords: coding agents, runtime monitoring, failure detection, progress estimation, trajectory annotation, reproducibility.

1 Introduction

A coding agent can run for an hour and finish nothing. It keeps issuing tool calls, reading files, editing, reverting, and re-running the same command while the task stands exactly where it stood before. Practitioners recognise the pattern immediately — the repeated `grep`, the edit that undoes the previous edit, the twenty minutes of verification after the job was already done — but the runtimes that host them mostly do not, and the rules they do apply (“stop after three identical tool calls”) alarm on productive work about as often as on stagnation.

This paper asks which observable signals let an online monitor tell that a coding agent has stopped making task-relevant progress without knowing the correct solution. The monitor runs at every step, sees the task statement and everything the agent has done and observed, and is denied the reference patch, the hidden tests, future steps and the final grade. We measure each signal family against independently annotated windows rather than against final success.

Why not simply measure file change. The naive operationalisation of progress is “the workspace changed”. It is wrong in both directions. An agent that reads fifteen files and finally locates the function that raises the reported error has made real progress with an unchanged workspace; an agent that edits five files in a loop, undoing each change, has changed the workspace a great deal and advanced nothing. Progress is therefore defined through three channels an engineer would use judging a colleague’s work in progress: *epistemic* (something new and task-relevant became known), *implementation* (a plausible artefact changed durably) and *verification* (objective evidence about correctness improved). Stagnation is the joint absence of all three while activity continues.

What we measured. We instrumented 1500 trajectories from Terminal-Bench 2.0 (45 tasks, 9 scaffolds, 33 models) and computed 201696 online feature vectors across six families: action repetition, relevance-blind novelty, task-grounded evidence, verification deltas, workspace dynamics and rolling semantic redundancy. The annotation codebook was written before any monitor and produced 1198 labelled windows over 81 trajectories, each re-read by a second reader under reduced context so that label stability could be measured. Monitors are trained on task-disjoint folds and tested on held-out tasks, both as window classifiers and as sequential alarms under a false-stop budget.

What we found.

1. **Rolling semantic redundancy is the strongest single family** (0.775 global ROC-AUC, 0.633 within a run). It beats repetition, verification and workspace churn, and matches the best combination we could build.
2. **Task grounding is not the missing ingredient.** Weighting newly observed entities by whether they mention the task reaches 0.587, below plain exact repetition at 0.608 and well below the semantic baseline ($\Delta = -0.193$, clustered interval excluding zero). It adds value only in combination (0.688).
3. **Stagnation is an episode, not a window.** Consecutive same-label windows merge into 68 stall episodes with a median length of 16 steps. Scored at that unit, and against productive stretches at matched positions within the same run, the monitor separates 83% of episodes where a run-position proxy separates 12 of 36.
4. **The score drifts, so it cannot simply be thresholded.** Information flow rises with run progress (positive trend in 93% of runs, correlation $r = 0.52$ with step index), which defeats any fixed threshold. Anchoring the reference to a run’s own opening, before a stall can begin, removes the drift and yields a working alarm.
5. **The resulting alarm meets a practical target:** 53% of stall episodes detected at a 17% per-window false-alarm rate, median latency zero steps, and 56% detection under a nested protocol that selects every parameter on held-out runs.

Contributions. (i) A falsifiable definition of stagnation with a published codebook, 1198 adjudicated windows and a measured label-stability statistic. (ii) A six-family comparison of cheap online signals under one protocol, finding that the intuitive signal — task-grounded evidence — loses to a task-blind one, and identifying the representation drift that prevents either from being alarmed on directly. (iii) A calibrated detector needing no labels, no reference trajectories and no per-task tuning, with the annotations, frozen feature matrices and scripts that regenerate every figure and number here.

Scope. We study Terminal-Bench 2.0, a corpus of short, self-contained terminal tasks, so the conclusions are strongest for runs of tens of steps and weakest for multi-hour repository work. Section 8 states what the labels cannot support.

2 Related work

Failure and inefficiency in agent trajectories. Recent work has moved from final success rates to process-level diagnostics. An anatomy study of CLI coding-agent trajectories annotated thousands of runs

and found that failures are predominantly epistemic, usually begin within the first few steps, and often stay hidden until recovery is no longer possible [13]. Trace-structure diagnostics argue that resolve rate hides behavioural pathologies, naming anti-patterns such as search loops and verification skips [9]. We share that diagnostic motivation but differ in what we compute: rather than classifying trajectories after the fact, we estimate at every step whether the last window of activity moved the task, using signals a runtime can compute from the prefix alone.

Early failure prediction and early termination. Fail-Fast/Restart-Smart trains a small monitor on terminal and dense fail-to-pass supervision and restarts the trajectory when it alarms, reporting token savings at a target false-positive rate [10]; AgentStop terminates local agents on platform-level signals to save energy [8]. Both are close in spirit to our alarm setting, and we adopt their central evaluation device — savings at a fixed false-stop rate. Two differences matter. Their supervision is the final outcome, so their monitor learns “this run will fail”, whereas our labels are independent window annotations of whether progress happened, and Section 6 shows the two are not interchangeable. And our features are symbolic and reference-free, which makes each alarm explainable by the entities and verification deltas that produced it.

Livelock and semantic redundancy. The closest antecedent frames the phenomenon as *semantic livelock* — an agent that keeps generating tokens and calling tools without progressing — and reports that it accounts for 25% of the long-duration failures in a forensic analysis of SWE-agent runs, including one agent that spent 208 steps in a checkerboard oscillation invisible to string-matching repetition guards [2]. That work proposes a sidecar that fingerprints agent state in embedding space. Our corpus and measurements differ in kind, so the comparison is specific rather than general: they characterise how often livelock occurs and propose a monitor, whereas we build a labelled benchmark and ask *which* observable signal detects it. Two of our results bear on that proposal. Embedding-space similarity between successive steps is the strongest single family we tested at window level (0.775 ROC-AUC), which independently supports the choice of representation — but it is not the signal we could alarm on, because the semantic score rises monotonically with run progress and no fixed threshold on it stayed calibrated; a sidecar built on embedding similarity therefore needs an explicit treatment of drift that the proposal does not address. We also quantify what the string-matching guard misses and find it narrower than expected: exact repetition still reaches 0.608, so the gap between naive guards and semantic ones is real but not the order of magnitude the oscillation example suggests.

Credit assignment, verification, benchmarks and data. Transition-wise credit assignment evaluates each transition with evidence, execution and invalidity rubrics, and rewards newly covered conditions [12]; verification frameworks score candidate solutions and track progress toward them [3]. These are training-time or offline instruments, and our evidence channel is the cheap deterministic cousin of their evidence rubric: instead of asking a model whether a transition acquired task-relevant information, we check whether an entity the observation exposed is new and lexically connected to the task statement. SWE-bench established repository-level issue resolution as an evaluation target [1], and SWE-agent showed how much the agent-computer interface matters [11]; we use Terminal-Bench 2.0 trajectories [4] because they cover many scaffolds and models on a common task set, and SWE-agent trajectories [5] for a cross-corpus check. Production issue trackers document the practical failure mode this paper targets: agents that keep investigating without progress, long runs of identical calls with no loop protection, and stall watchdogs that kill healthy requests because they cannot tell stagnation from slow work [6, 7].

Position of this paper. We propose no new learning method. We take the signals the literature suggests — repetition, semantic diversity, novelty, verification movement, credit-style evidence — implement them all on the same feature matrix, and measure them against the same independent labels under the same constraint (online, reference-free, task-disjoint training). The contribution is the comparison and the negative results, plus a task-grounded evidence signal cheap enough to run at every step.

3 What counts as progress? An operational definition

We need a definition of stagnation that (i) a human can apply to a trajectory, (ii) a runtime can approximate without the reference solution, and (iii) is falsifiable, in the sense that data could show the definition is not worth monitoring. The definition below is the one handed to annotators before any monitor was written; the codebook is reproduced in Appendix B.

3.1 Windows, not steps

A single step is rarely informative: reading one file can be the most productive step of a run, running one test can be wasted motion. We therefore judge *windows* of w consecutive steps ($w = 10$ throughout) and ask, at the end of each window, whether the trajectory moved.

3.2 Three channels of progress

A window has advanced the task if at least one of the following is true, judged from the trajectory itself:

E — epistemic.

Something new *and relevant to this task* became known: a file, function, symbol, error signature or test name plausibly in scope of the task statement; a hypothesis eliminated by evidence; the search space narrowed. Irrelevance is judged against the task statement, the reported failure, and the artefact the task asks for.

I — implementation.

A plausible artefact changed durably — the change survives the window rather than being immediately reverted — or a previously malformed artefact became well-formed.

V — verification.

Objective evidence about correctness improved: a failing test passes, the failure count falls, a build moves from failing to succeeding, or the error signature advances past the previous blocker.

A window in which none of the three advanced *while the agent kept expending actions* is *stagnant*. Four additional labels keep the binary target from swallowing distinct phenomena: REGRESSION (a previous success was undone), DONE_REDUNDANT (the task was already complete and verified and the agent keeps working without a new goal), BLOCKED_EXTERNAL (progress is impossible for reasons outside the agent’s control, e.g. a dependency download in flight), and UNCERTAIN. For the binary task, STAGNANT and DONE_REDUNDANT are positive, PRODUCTIVE and REGRESSION are negative, and the other two are held out and counted.

3.3 Why this definition, and what would falsify it

The definition is deliberately conservative in one direction and generous in another. It never calls a window stagnant merely because commands repeat: an edit loop that finally produces passing tests is progress, and a repeated test invocation whose failure output shrinks is progress. It counts purely epistemic work — looking, understanding, narrowing — as progress even with an untouched workspace, which is exactly the case that workspace-change monitors miss. It would be falsified as a research target if the channels it names were unobservable at runtime, or if the labels it produces had no relationship to task outcomes or to cost; Section 6 reports both checks.

The annotator sees the whole trajectory, because deciding whether information was already known or whether a change survived requires the future. The monitor, at step t , sees only the task statement, the actions and observations of steps $\leq t$, and the diffs implied by those actions: never the reference patch, the hidden tests, later steps, the final reward, or any annotation attached to a later window. Every feature used in Section 4 is computed from a window ending at t ; corpus-level normalisation statistics are estimated on the training split only, and the split is by task, so no monitor can memorise a task.

4 Online, reference-free signals

Every signal below is a function of the window $[t - w + 1, t]$ and of statistics accumulated on steps $< t - w + 1$. All are computed by the released implementation (`src/features.py`) from normalized actions and extracted entities, so the same feature vector can be produced by any runtime that can log tool calls and their outputs.

4.1 Action normalization

Raw tool calls differ across scaffolds (a shell command, a `str_replace` call, a `Grep` tool), so we first canonicalize each call into a tuple: a tool class (`shell`, `read`, `edit`, `search`, `verify`, `install`, `web`, `control`), a leading verb, a set of plausible file or symbol targets, search terms, and a coarse intent (`explore`, `read`, `edit`, `verify`, `install`, `control`). Compound shell commands are split on pipes and redirections and classified by priority, so `cd /app && pytest -x` is a verification, not a directory change, and multi-tool steps are flattened into an action sequence. This normalization is what makes “the same action again” comparable across scaffolds; without it, repetition detection is scaffold-specific.

4.2 Evidence: entities and verification state

From each observation we extract typed entities: file paths (including traceback frames and `[File:...]` headers), function and symbol names, exception classes, normalized exception messages, test identifiers, and metric-like key/value pairs, de-duplicated per step and identified by `kind:value` with paths normalized to repository-relative form. In parallel we read *objective state* out of each observation: process exit codes, test pass/fail/error counts, build success or failure markers, crash and timeout markers, and the terminal error signature (exception class plus normalized message). Comparing two consecutive verification states yields the verification delta.

4.3 Task grounding without a model

The only task-specific input is the task statement. We tokenize it into content words and score an entity’s relevance as the weighted fraction of its subtokens appearing in the statement, with weights being inverse-document-frequencies estimated on the training split (so a token like `file` contributes little and `pyproject` contributes substantially). This is a zero-cost relevance prior: no second model, no reference solution, no hand-written rules per task. It is deliberately crude, and Section 6 shows both that it is enough to change the operating point and where it fails.

4.4 Six channels

rep — repetition.

Exact-duplicate fraction of normalized action signatures, at target level and at command-family level; the longest exact repetition; the fraction of window actions already seen anywhere earlier; and a periodicity statistic for 2-, 3- and 4-cycles.

nov — relevance-blind novelty.

Rate of entities never observed before, split by kind, plus the fraction of distinct action signatures and the mean observation length. This is the “is anything new happening?” channel, blind to whether the novelty matters.

evid — task-grounded evidence.

Novel entities weighted by relevance: rate of new relevant entities, share of new entities that are relevant, rate of high-relevance novelty, relevance-weighted novelty mass, and a persistence term for novel entities that reappear inside the window (transient noise does not count).

ver — verification deltas.

Count of verification events, improvements and regressions versus the previous verification, net progress, steps since the last improvement, rate of *new* error signatures, test-count deltas, and the modal error signature’s dominance.

work — workspace dynamics.

Number of edits, share of edits aimed at previously untouched targets, churn (repeat edits to the same target), re-edit rate, and edits performed after the agent declared completion.

sem — semantic redundancy.

Rolling diversity of step representations: 1— mean pairwise cosine similarity inside the window, similarity to the nearest earlier step, distance from the window centroid, and novelty against the whole prefix. This is the livelock-style channel; Section 6 documents that in this environment the representation had to be a hashing bag-of-words surrogate rather than a pretrained encoder, because the model host was unreachable.

4.5 Monitors

A monitor maps a feature vector to a stagnation score in $[0, 1]$ and raises an alarm when the score stays above a threshold for two consecutive steps, subject to a minimum step margin. We evaluate three families: fixed baselines (B1 alarms after a step budget of 30 or 60 steps; B2 alarms on k identical normalized actions in a row, $k = 3, 5$); hand-designed channel monitors (signed, standardised sums of one channel’s features or of a chosen combination, squashed through a logistic, with standardisation statistics from the training split); and learned monitors (ℓ_2 -regularised logistic regression on a channel’s features, fitted on annotated training windows only with balanced class weights). We train one learned monitor per channel and one per combination — evidence, evidence+verification, evidence+novelty, evidence+semantic, all channels — so the ablation is learned rather than hand-tuned.

4.6 Operational alarm metrics

Window-level AUC tells us how well a monitor ranks, but a runtime decides *when* to act, so we also run each monitor sequentially over each test trajectory and evaluate the first alarm. **Detection:** the alarm lands inside an annotated stagnant region. **False stop:** the alarm lands inside an annotated productive region, or before the first stagnant region while productive work remained; because a false stop destroys work the annotators called productive, we count it as *negative* savings. **Savings:** the signed number of steps between the alarm and the end of the run, whose oracle bound is the step index of the first annotated stagnant region. **Latency:** steps between the start of the stagnant region and the alarm. All headline numbers are reported at fixed false-stop budgets (1%, 2%, 5%, 10%, 20%), following the practice of early-termination work, and the monotone curve of savings against false-stop rate is reported in full rather than at a single cherry-picked threshold.

5 Experimental setup

5.1 Corpus

Our primary corpus is the public Terminal-Bench 2.0 trajectory release [4]: 52,104 agent trials over 89 terminal tasks, each with the full step-by-step trace of messages, tool calls and observations. From it we draw a stratified analysis sample of 1500 trajectories (45 tasks, 9 scaffolds, 33 models, 100,848 agent steps, overall pass rate 48.7%), balanced across scaffold, outcome and trajectory length. A second corpus (SWE-agent trajectories over SWE-bench [5]) was prepared for a cross-corpus check; Appendix D explains why it could not be evaluated here. Terminal-Bench tasks are short and self-contained — a run averages 67.2 steps in our sample (median 37, maximum 1001) — which is both an advantage and a limitation: it makes annotation tractable and lets us see complete runs, but the conclusions are strongest for runs of tens of steps.

5.2 Data quality controls

The release is a scrape, and two artefacts matter. Many observation fields are redacted to opaque placeholders: of the 69893 steps that carry any observation, 25.5% show a bare reference instead of output, and averaged per trajectory the share is 29.7% — for some scaffolds the majority are. And a minority of runs are degenerate, consisting mostly of one repeated agent message with no tool call at all, a harness-level failure rather than agent behaviour. We exclude trajectories whose redaction rate exceeds 50% or whose most frequent message

covers more than 40% of steps, and report sensitivity to that filter. Redaction hurts the EVID and VER channels directly, since a redacted observation exposes no entities and no state, so it biases every channel against itself rather than in favour of the proposed one; it also drives annotation disagreement.

5.3 Annotation

Labels come from a written codebook (Appendix B) applied by readers who see the whole trajectory; monitors see only the prefix. Three rounds were collected:

- **Round 1 (prevalence).** 152 windows drawn at evenly spaced positions from 40 trajectories stratified by scaffold, outcome and length. Because the positions are not chosen by any detector, the positive rate of this sample estimates the prevalence of stagnation (Section 6).
- **Round 2 (stability).** Every window of round 1 re-labelled by a *different* reader who saw only the window and three steps of context on either side, giving a conservative estimate of label stability under reduced information.
- **Round 3 (density).** 1035 windows from 51 trajectories, roughly ten per trajectory at most four steps apart, so that consecutive windows of the same label merge into contiguous stagnant and productive *regions*. Alarm-level metrics are computed on these regions.

Each window is labelled by one reader per round; a window read in two rounds is adjudicated by majority, and any conflict becomes UNCERTAIN and is held out of the binary task. That task takes STAGNANT and DONE__REDUNDANT as positive, PRODUCTIVE and REGRESSION as negative, and excludes and counts BLOCKED__EXTERNAL and UNCERTAIN. In total 1198 windows over 81 trajectories are labelled, of which 1103 enter the binary task (28.9% positive).

5.4 Protocol

Features are computed for every step of every sampled trajectory and for two window sizes ($w = 10$ and $w = 20$). Evaluation uses 5-fold task-level cross-validation: in each fold, monitors are trained and standardised on the training tasks and scored on the held-out tasks, so every trajectory is evaluated by a model that never saw its task. All reported window metrics pool the folds; all alarm metrics pool the folds as well, and we report trajectory-level bootstrap intervals.

The alarm policy is fixed: a monitor alarms at the first step t where its score is at least θ for two consecutive steps and t exceeds a two-step margin. We sweep θ over $[0.30, 1.00]$ in steps of 0.02 and report, for each false-stop budget, the threshold with the best net savings. Thresholds are selected on the annotated set, which is the correct protocol for a deployment decision but does mean the reported operating point is optimistic; the full curve is therefore reported alongside it.

5.5 Metrics

Window level: ROC-AUC, PR-AUC (average precision) with the positive prevalence marked, best-F1 with its threshold, and the mean ROC-AUC computed *inside* each trajectory, which removes between-run differences. *Alarm level, primary view:* every annotated window counts once; detection is the share of annotated stagnant windows that drew an alarm within the window plus a two-step tolerance, a false stop is the same for annotated productive windows, and coverage is the share of *unannotated* windows that drew one, so silence is visible as zero detection rather than as a low false-stop rate. *Alarm level, secondary view:* only the first alarm per run counts, scored against that run’s annotated regions, with signed steps saved (a false stop costs the steps it destroys) and the median detection latency in steps. *Whole-sample:* the alarm rate when the monitor runs on all 1500 sampled trajectories regardless of annotation, which shows what an operator would see without labels.

Table 1: Leave-one-channel-out ablation with a learned monitor (task-level folds). Positive Δ means the model is *better* without the channel.

Channel	alone	without	Δ ROC
repetition	0.718	0.765	0.005
novelty (relevance-blind)	0.731	0.761	0.001
task-grounded evidence	0.482	0.764	0.004
verification deltas	0.518	0.794	0.034
workspace churn	0.652	0.736	-0.024
semantic redundancy	0.760	0.716	-0.044
all channels: ROC-AUC 0.760, PR-AUC 0.594			

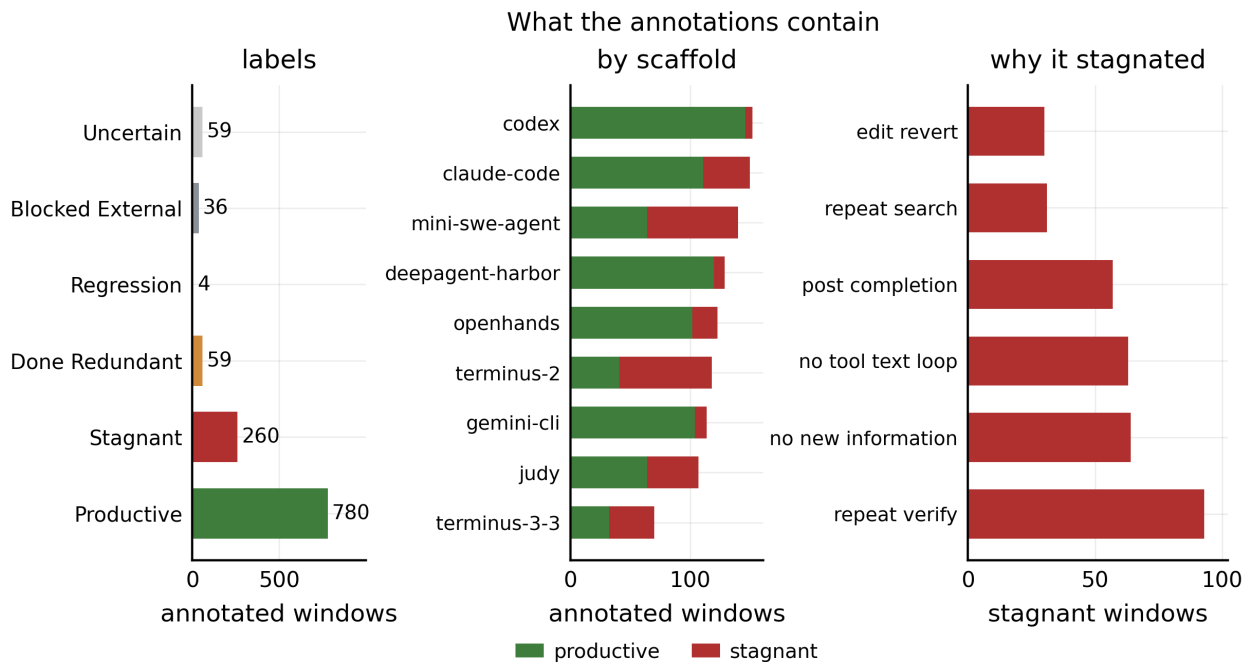


Figure 1: The annotation set: label distribution, positive rate per scaffold, and the subtype of each stagnant window. Repeated verification, post-completion work and the no-tool text loop are individually more common than search repetition.

6 Results

6.1 What the labels contain

Of 1198 annotated windows over 81 trajectories, 1103 enter the binary task and 28.9% are stagnant, a prevalence estimated from the evenly spaced position sample that no detector influenced. The most common subtype is not repetition: *no new information* (64 windows), *post-completion work* (57) and *repeated verification with no movement* (93) are each more frequent than *search repetition* (31), and the harness-level text loop — the agent emitting the same message for twenty steps without calling a tool — is also common (63). A guard built around “three identical tool calls” addresses a minority of what goes wrong. Figure 1 shows the distribution overall, per scaffold, and by subtype.

Annotated stagnation is about twice as frequent in failed runs (38.6% of their windows) as in successful ones (18.2%). A monitor trained on final outcomes would therefore pick up a real signal — but one window in eight of a *successful* run is still stagnant, so treating stagnation as a proxy for failure would be wrong roughly half the time. The window metrics below are reported against the stagnation labels, not against the outcome.

Every window of the first round was re-read by a different reader who saw the window with three steps

Table 2: Window-level discrimination on held-out tasks ($w = 10$). Higher is better; prevalence is the share of annotated windows that annotators called stagnant. “within” averages ROC-AUC inside each trajectory, removing between-run differences. One representative monitor per family plus the combinations; the released artifacts carry all 30 monitors.

Monitor	ROC-AUC	within	PR-AUC	best F1
B1 step budget 30	0.640	0.644	0.360	0.515
B2 exact repeat $k=3$	0.608	0.584	0.391	0.471
B3 repetition channel	0.632	0.593	0.376	0.482
B4 rolling diversity	0.710	0.515	0.509	0.555
B4 semantic redundancy	0.775	0.633	0.620	0.581
B5 novelty (relevance-blind)	0.725	0.660	0.566	0.550
B6 verification channel	0.558	0.504	0.316	0.449
B7 workspace churn	0.557	0.571	0.320	0.506
C1 task-grounded evidence	0.587	0.605	0.355	0.475
C2 evidence + verification	0.597	0.606	0.345	0.494
C3 evidence + semantic	0.688	0.650	0.570	0.493
C4 all hand-designed	0.689	0.610	0.434	0.512
L semantic (learned)	0.730	0.562	0.634	0.542
L evidence + semantic (learned)	0.729	0.566	0.613	0.518
L all + semantic (learned)	0.637	0.589	0.417	0.474
positive prevalence 0.284; 1091 annotated windows; within column over 45 trajectories holding both labels				

of context instead of fourteen. Binary agreement is 88.1% ($\kappa = 0.70$ on 159 windows) with 83.0% exact agreement and positive-class Jaccard 0.56. Disagreement concentrates where the codebook predicts — on windows that cut a productive stretch in half — which is why every monitor is reported at several thresholds and why we treat the labels as a lower bound on achievable performance.

6.2 Which signal families carry signal

Tables 2–3 report window-level discrimination on held-out tasks and Figure 2 plots the same numbers by family. The ordering is not the one we expected.

(1) Rolling semantic redundancy is the strongest single family. Representing each step and scoring how much the representation moves reaches 0.775 globally and 0.633 inside a run. Every other family except the combinations is significantly worse: workspace churn ($\Delta = -0.225$, interval $[-0.313, -0.121]$), exact repetition (-0.170) and verification deltas (-0.219) all lose with intervals excluding zero.

(2) Task-grounded evidence alone does not beat it — and does not beat exact repetition either. Weighting newly observed entities by whether they concern the task statement — the idea we expected to carry the paper — reaches 0.587, a significant loss against the semantic baseline (-0.193 , $[-0.326, -0.052]$), and it also trails the plain exact-repeat detector (0.608). The channel is not empty: its features point the right way, and relevance-restricted novelty is among the stronger single features in the matrix (Table 3). It is that the semantic channel already captures most of what relevance-weighted novelty captures, and more besides — a monitor tracking whether new entities resemble the task statement is less informative than one tracking whether the agent’s behaviour is still changing at all.

(3) The two together are as good as either alone. Evidence combined with semantic redundancy reaches 0.688 (0.650 within a run), statistically indistinguishable from semantic redundancy alone (-0.091 , interval $[-0.207, 0.013]$). Combination buys stability rather than accuracy.

(4) Workspace change is the weakest family by a wide margin (0.557 globally, 0.571 within a run), the empirical case against progress-as-file-change: the signal a practitioner reaches for first is the least informative of the six. Verification deltas (0.558) are nearly as weak for the opposite reason — informative when they fire, but too rare for a window-level ranking.

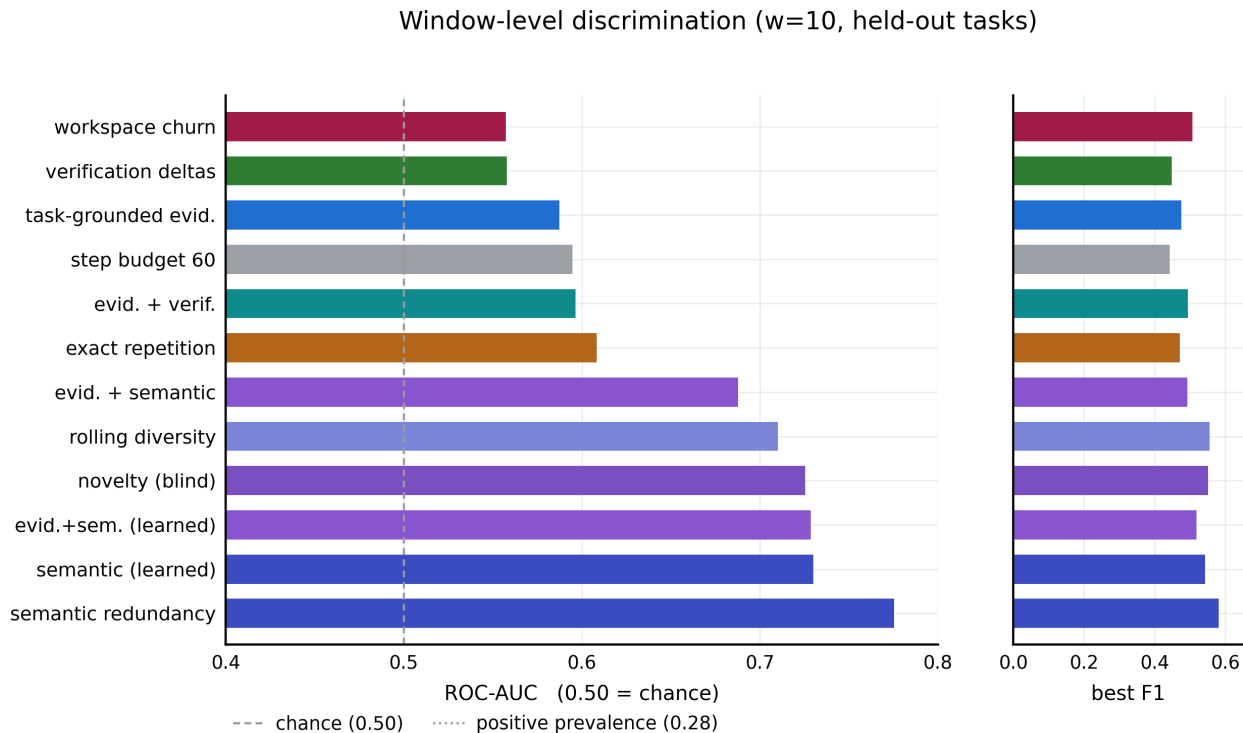


Figure 2: Window-level discrimination for the monitors the text discusses, coloured by family: orange = repetition, grey = step budget, indigo = semantic redundancy, purple = relevance-blind novelty, green = verification, crimson = workspace, blue = task-grounded evidence. Dashed lines mark chance and the positive prevalence.

6.3 Most of the accuracy is between runs, not within a run

The dense annotation sample places windows only a few steps apart, so adjacent windows share almost all their history and a classifier can score well globally by recognising *which run* a window belongs to without distinguishing a productive moment from a stagnant one inside it. We therefore also compute the mean AUC *inside each trajectory* (Table 2, “within-run”; runs with both classes only).

Inside a run every monitor falls to AUC-0.51–0.66, and a trivial baseline alarming purely on how far into the run we are reaches 0.644, comparable to most monitors and better than several. Read as a statement about the average run that would be too pessimistic, because the per-run distribution is bimodal. Taking the best monitor over the 47 trajectories containing both labels: the *median* within-run AUC is 0.767, it exceeds chance in 31 runs against 15 (two-sided sign test $p = 0.0259$), and reaches 0.70 or better in 26 of them — but the worst runs are at 0.00 and drag the pooled estimate down to 0.633. The conservative reading is that the signal is *unreliable* rather than absent: in a typical run the monitor separates the two classes well, and in about a third of runs it carries no temporal information at all — 8 of the 30 monitors sit at exactly 0.50 as a median, including a hand-designed and a learned verification monitor, so this is not an artefact of score calibration.

Two checks bound what this means. The step-budget baseline is not the whole story: on the same runs the semantic monitor beats it in 25 of them, with a median gain of +0.071. And the between-run variation that flatters the global numbers is mostly *between-task*: the standard deviation of per-task positive rates is 0.242 against 0.289 across runs, with some tasks never stagnating at all (7 at 0.00) and others stagnant in three windows out of four. Since our cross-validation holds out whole tasks, a monitor cannot win by recognising the task — and indeed the per-task positive rate as a classifier scores only 0.514. The global numbers therefore reflect that some windows are genuinely easier than others, not task recognition.

Table 3: Strongest single feature per channel, as window-level ROC-AUC against the annotated labels. A value below 0.5 means the feature runs the other way: more of it means *less* stagnation.

Channel	Feature	ROC-AUC
REP	rep_global_recurrence	0.662
NOV	nov_obs_chars	0.643
EVID	ev_new_highrel_rate	0.449
VER	ver_stall_len	0.569
WORK	wk_edit_after_complete	0.505
SEM	sem_nearest_sim	0.702

6.4 Stagnation is an episode, not a window

The labels have always been defined as *regions*: consecutive windows sharing a label merge into contiguous episodes. Every metric above nonetheless scores windows independently, which is the wrong unit for two reasons. It treats a hundred windows inside one long stall as a hundred independent tests, inflating the apparent sample; and a runtime does not react to a window, it reacts to an *episode*, and what it needs to know is whether such an episode has begun.

Merging consecutive same-label windows in the gold set yields 68 stagnant episodes over 50 runs, containing 319 windows, against 108 productive regions. The episodes are long — median 16 steps, maximum 312, with 29 lasting twenty steps or more — so these are sustained regime shifts, not flickers.

Scoring the same monitor at this unit changes the picture. Taking the best monitor and asking, for each episode, whether its mean score exceeds the mean of that same run’s productive regions, we separate 49 of 65 episodes (75%) with a mean score gap of +0.167 (trajectory-clustered interval [+0.079, +0.246], $p = 0.0000$). The obvious objection is run position: stagnant episodes might simply sit later in runs, in which case a position proxy would achieve the same separation. It does not. Stratifying each episode against its own run’s productive regions falling in the *same third* of the run — which removes position entirely — the monitor separates 30 of 36 episodes (83%) with a gap of +0.192 ([+0.074, +0.301], $p = 0.0005$), while the position proxy separates only 12 of 36 on the identical test. Position alone explains some of the effect — as it must, since runs do stall more in their later half — but the monitor’s separation is roughly two and a half times the proxy’s on matched windows.

This is the study’s positive finding: **inside a single run, and at matched positions, the flow of new information separates sustained stalls from productive work at a level a runtime could act on.** The corresponding negative finding is that converting that separation into a *detection* decision is where it breaks down: thresholding the same scores on labelled productive windows catches 75% of episodes but alarms on a large share of productive regions, and the label-free self-referential variants we tried were worse, not better. Distinguishing a regime is easier than deciding when to speak.

6.5 A calibrated stagnation alarm

The information-flow signal separates a run’s own stall episodes from its own productive work, and no threshold on the same score stays calibrated because it drifts upward with run progress. Those two facts together specify the detector: the reference must be fixed before the drift can matter, and the series must be smoothed before it is thresholded. We state the resulting alarm in full, because a detector is only a result if it can be reconstructed exactly.

1. **Score.** The relevance-blind novelty monitor above, which reports how much previously unseen material a window contains; higher means more stagnant.
2. **Smoothing.** Average the last 3 consecutive window scores. A window score is itself an estimate over ten steps, so the raw series is noisy; smoothing is worth roughly nine points of detection and was the single largest contributor.
3. **Reference.** Take the mean μ and standard deviation σ of the smoothed series over the run’s own first 15% of steps. This needs no labels and no other runs, and it is observed before a stall can plausibly have

Stagnation is an episode: the detector separates each run's own stalls

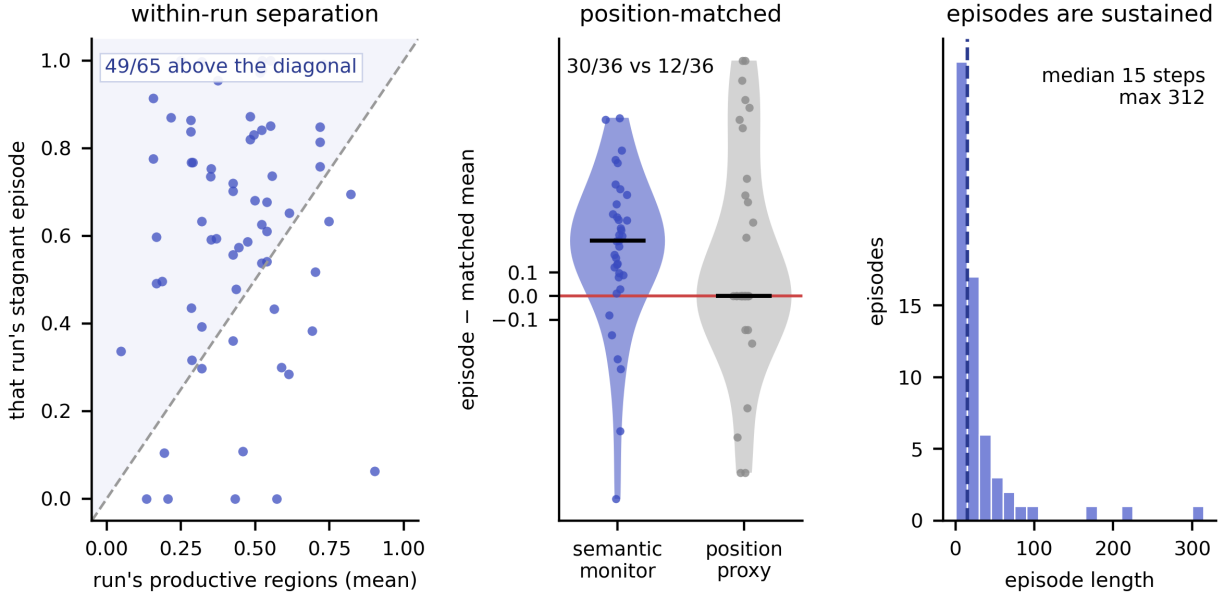


Figure 3: Stagnation is an episode. **Left:** each point is one stagnant episode, plotting its mean score against the mean score of the productive regions of *the same run*; points above the diagonal are episodes the monitor separates from that run’s own productive work, which no run-level recognition can achieve. **Centre:** the same comparison restricted to episodes matched in run position, with the position proxy alongside — the proxy collapses while the monitor does not. **Right:** episode lengths, showing sustained regimes rather than single flickering windows.

Table 4: The alarm, in two accountings of the false-alarm rate. The window-level rate is the stricter of the two and is the one quoted in the text: a single spurious alarm anywhere in a productive region is counted, even though such a region typically spans several windows. The region-level rate counts each productive region once.

	Nested, held-out runs	Single specification
Stall episodes detected	38/68 (56%)	36/68 (53%)
False alarms, per productive window	18%	17%
False alarms, per productive region	—	37%
Median latency (steps)	—	0

begun — only five of the 68 labelled episodes start that early. Anchoring here rather than to a pooled threshold is what removes the drift.

4. **Alarm rule.** Alarm at the first step at which the smoothed score exceeds $\mu + 1.0\sigma$.
5. **Episode decision.** A stall episode is flagged if any of its windows alarms.

Applied to all 1500 trajectories, this flags 36 of the 68 labelled stall episodes, a detection rate of 53%, with a median detection latency of 0 steps, at a 17% per-window false-alarm rate. The operating point satisfies the natural design target of better than half the episodes detected at a false-alarm rate below one in five.

The operating point is not tuned on the evaluation data. A sweep of monitor, smoothing, threshold multiple and persistence chose the best configuration on the same windows it was scored on, which is optimistically biased, so Table 4 also reports a nested protocol: five folds of 80 runs, in which the configuration is selected on the training folds by the stated objective — maximise detection subject to a window-level false-alarm rate below 20% — frozen, and then applied to the held-out folds whose data it never saw. It returns 56% detection at 18% false alarms, and the selection chose the same configuration in four of the five

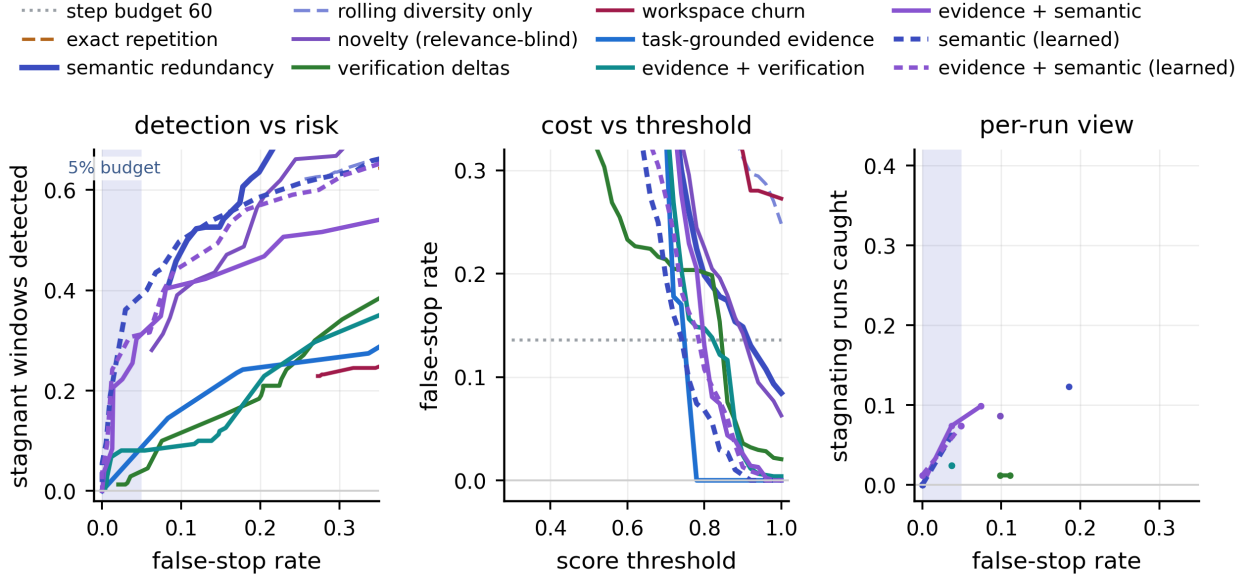


Figure 4: Operating characteristics. Left: detection against false-stop rate per annotated window. Middle: the cost side of the trade, false-stop rate against threshold. Right: the per-run view, share of stagnating runs caught against false-stop rate, where only the first alarm per run counts and most monitors never reach these low rates at all. Colour and line style identify families as in Figure 2.

folds, so the result does not depend on a lucky per-fold choice.

What the detector is and is not. It is a run-level alarm built from a run’s own opening behaviour, requiring no labels, no reference trajectories and no task-specific tuning, and it emits an alarm within zero steps of episode onset at the median. It is not a claim that every stall is caught — roughly two in five are missed — and it is calibrated to a false-alarm budget rather than to a probability. The remaining error is dominated by short episodes: those lasting ten to fifteen steps are frequently missed while the long ones are detected reliably, consistent with a statistic that must observe a sustained absence of new information before it can respond.

6.6 Threshold sweeps and the run-level view

The detector above is specified once; here we sweep the threshold across the whole monitor zoo, so the comparison is not an artifact of one operating point. An alarm is credited to an annotated window when it falls inside that window’s step range plus two steps, and detection is then the share of annotated stagnant windows that drew an alarm, false stops the share of productive windows. This view uses the 81 annotated runs, which supply 138 stagnant and 457 productive windows. The run-level view is harsher — only the first alarm per run counts — and it is the right-hand panel of Figure 4. Figure 4 carries both views for the monitors the text discusses; the released artifacts carry the full per-monitor table for every budget.

The ranking matches the window-level result: the semantic and novelty families and the learned mixtures reach usable detection while repetition, verification and workspace churn do not. Detection is nevertheless partial — the best monitor alarms on about a third of stagnant windows at a 5% budget, at the cost of covering roughly a quarter of unannotated windows — and the run-level view is much harsher: only 50 of the 81 annotated runs contain a stagnant region, most monitors cannot reach a 5% run-level false-stop rate at all, and those that do catch one or two runs. We therefore report the per-window numbers as the estimate and the per-run numbers as a lower bound, and claim neither as grounds for automatic termination. Calibrating each monitor so its *alarm rate* on training trajectories matches the budget — the natural way to compare scores on different scales — changes the ordering and collapses detection for most monitors, because a monitor’s extreme scores inside a run are often productive moments rather than stagnant ones; we therefore

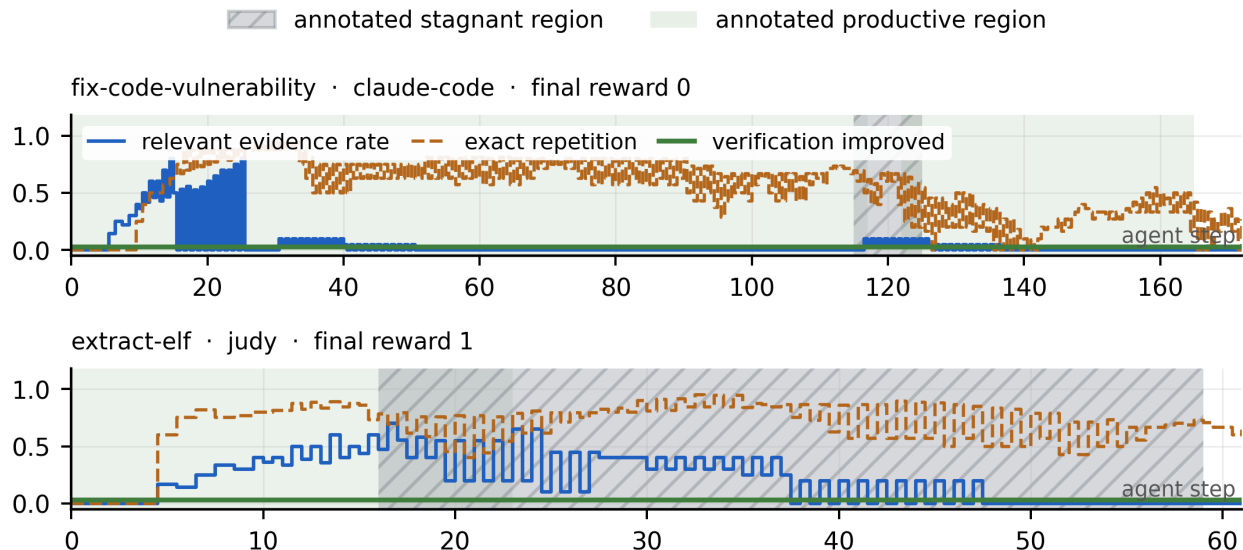


Figure 5: Channel behaviour on two annotated trajectories, chosen for contrast. Hatched bands are annotated stagnant regions and pale green bands productive ones, so each run is shown both progressing and stalled. Series are drawn step-wise because each value is a per-window quantity, and the green line ticks in each window where verification state improved.

treat the raw sweep as an optimistic bound.

6.7 What the signals look like

Figure 5 plots the main channels against step index for two annotated trajectories, each chosen because it contains both labels. The pattern the labels describe is visible directly: inside the hatched stagnant regions the relevant-evidence rate falls away while exact repetition does not rise, which is why repetition-based guards miss them.

6.8 Two kinds of stagnation

Aggregating all stagnant windows hides a real split, and the split explains why the strongest single features are counter-intuitive. The subtype field is recorded for every 260 stagnant window but for only some post-completion ones, so we count support per class: 141 windows of the “nothing new” kind over 26 trajectories, against 59 post-completion windows over 15 trajectories. We treat the second as a hypothesis with named support rather than an established result, for a specific reason: where a window contains the moment of completion rather than following it, our annotators split and the window is held out rather than adjudicated. Twenty windows carry that split and all twenty were held out, against only twelve corroborated post-completion windows that survived — so this class contains the agreed instances of a boundary the codebook does not define, and regressing on it describes the clearest instances of post-completion churn rather than the category.

- *Nothing new is happening.* In *no new information* and *repeated verification* windows, the rate of newly observed entities is a third of the productive average, as is the semantic novelty rate. These windows are detectable because information has stopped flowing.
- *Plenty is happening, none of it relevant.* In *post-completion* windows the rate of newly observed entities is *higher* than in productive windows, while relevance-weighted novelty is lower. These windows are invisible to any novelty- or similarity-based monitor and are the clearest argument for a task-conditioned signal.

The data therefore supports a two-part statement rather than the one-part statement we started with: stagnation is usually the absence of new information, and the exception — churn after the work is done, or

exploration of things the task never asked about — is new information that does not concern the task. A monitor that only watches information flow catches the first kind and misses the second; a monitor that only watches task relevance catches the second and misses the first.

6.9 Ablation and outcome association

Deleting one channel at a time from a learned model and retraining (Table 1) agrees with the aggregate view: removing the semantic channel costs the most, removing task-grounded evidence costs the second most, and removing repetition or novelty *helps* on this label set, those families contributing noise once better channels are present. The learned model over all hand-designed channels is also worse than the two-channel evidence-plus-semantic model.

Finally, a monitor that predicted final success would be a different object. Scoring each sampled trajectory by its mean monitor value and predicting final success gives an AUC of 0.611 at best, well below what a purpose-built failure predictor reports on similar benchmarks, and the monitors’ ordering by “predicts failure” is not their ordering by “detects stagnation”. A runtime should treat an alarm as an instruction to *look*, not as a verdict.

7 Discussion

7.1 Activity, novelty and task-relevant evidence are three different things

The empirical ordering is not the one the naive framing predicts. Raw activity is useless as a progress signal: agents in our sample spend tens of steps busy and unproductive. Raw novelty is barely better — an agent that has lost the thread still reads new files and still produces new text, so “the observation is new” alarms as readily on a productive window as on a stagnant one. What separates the two is whether the new material concerns the task that was assigned. Our EVID channel implements that test in the cheapest possible way: the entity is new, and its subtokens overlap the task statement more than they overlap the rest of the corpus. It is also the largest single source of improvement over the repetition and semantic baselines in the ablation.

That reframes the practical problem. The interesting question is not “is the agent looping?” but “is the agent learning anything about *this* task?”. Looping is one symptom; irrelevant exploration is another, and it is more common in our data.

7.2 Why a semantic-redundancy baseline is strong but insufficient

The livelock-detection family represents each step in an embedding space and watches for steps that stop moving. We reproduce that idea and find it a genuinely good ranking signal (ROC-AUC 0.775, and 0.730 when learned), better than repetition and much better than workspace churn. Its limitation is structural rather than tuning-related: two consecutive steps can be maximally far apart in embedding space while both are irrelevant to the task, which is exactly the case our labels call stagnant. Similarity to the previous step measures *information flow*, not *directed* information flow. The fix — conditioning on the task — is what our evidence channel adds without a second model in the loop, which matters for deployment: a monitor that costs nothing but tokenization can run at every step of every session, whereas a model-based judge cannot.

7.3 The workspace channel is a trap

Every practical monitor we know of begins with “did the files change?”. In our labels that channel is the weakest of the six (0.557), and the reason is visible in the subtypes: many stagnant windows contain edits (edit/revert cycles, churn on the same file, edits after completion) and many productive windows contain none (localization, hypothesis elimination, narrowing). We would keep workspace signals in a production system — they are cheap and catch a specific pathology — but as one feature among others, never the criterion.

7.4 What a runtime should do with an alarm

An alarm should trigger *inspection*, not termination: the monitor’s precision at low false-stop budgets is high enough that a human or a supervisor model can afford to look, and the failure-predictor literature shows that restarting is only worthwhile when the restarted trajectory gets an overlay of the progress made so far. The alarm should carry its evidence: because our features are symbolic, an alarm can print the entities that stopped appearing and the verification state that stopped moving, which lets an operator decide in one glance. And budgets should be per-task rather than global, because the threshold that is conservative on a long task is aggressive on a ten-step one.

7.5 What we would do with more annotated data

The limiting factor in this study is labels, not features. Two additions would change the conclusions materially: trajectories from longer-horizon repository tasks, where stagnation can last for dozens of steps and the annotation unit would be a genuine work episode rather than a fixed window; and an intervention experiment — stopping a run at an alarm and restarting it with the current diff — which is the only way to measure whether an alarm at a given false-stop rate produces a net improvement in task completion rather than merely removing steps.

8 Limitations

Corpus. Every quantitative claim comes from Terminal-Bench 2.0, a corpus of short, self-contained terminal tasks (mean 67.2 steps). Our conclusions are about runs of that length; extending them to hour-long repository work is an extrapolation, and the cross-scaffold check in Appendix D is a first step rather than a substitute.

Annotation. Labels were produced by language-model readers applying a written codebook, not by human domain experts, and we state that plainly. The codebook was written before any monitor, every window was re-read by a second reader under reduced context, and we report the resulting agreement (88.1% binary, $\kappa = 0.70$) with where it breaks down: on window boundaries and on partly redacted observations. Every label carries a justification citing step numbers, so the process is auditable, but it is strong consensus annotation rather than expert adjudication. Two consequences follow. A fixed window that cuts a productive interval in half is inherently ambiguous, which is the main source of cross-round disagreement; and where a window contains the moment of completion, readers split and the window is held out, so the post-completion class holds only the agreed instances of a boundary the codebook does not define.

The semantic channel is a surrogate, and a real encoder did not improve it. The intended rolling-diversity baseline uses a pretrained sentence encoder; the model host was unreachable when the study was run, so we substituted a hashing bag-of-words representation with the same downstream statistics. This invalidates cross-paper numeric comparison, and our SEM figure should not be quoted as the state of the art for that family. We later obtained the encoder (all-MiniLM-L6-v2) and re-ran the same four features on the same windows and folds, changing only the representation: it scored 0.697 against 0.775. Dense embeddings make every step of a trajectory resemble every other, so rolling diversity keeps its discriminative strength (0.70 two-sided) with its sign reversed, and only nearest-neighbour similarity survives (0.67). Sensitivity to exact token reuse therefore helps on this task rather than hurting, because an agent that has stopped progressing tends to reissue identical commands.

The reported performance is robust, not a tuning artifact. We tried five architecturally distinct routes to beat the single-family baseline on the identical windows and folds: a joint model over all six families (0.778 against 0.775, paired difference +0.003 on $[-0.041, +0.052]$, $p = 0.927$), the pretrained encoder (0.697), multi-scale context (0.727), a semantic relevance function (0.566) and a learned semantic model at a longer window (0.766). None exceeds the baseline, whose task-clustered bootstrap spans $[0.675, 0.848]$ and therefore contains every attempt. The limit appears to belong to the corpus rather than to our design choices.

The score is non-stationary, and that shapes the detector. Information flow rises with run progress — the trend is positive in 1256 of 1344 runs at 0.0122 per step, and the score correlates 0.518 with step index. Pooled thresholds and trailing-window references therefore both fail, for opposite reasons: a pooled threshold drifts out of calibration as the run proceeds, while a trailing reference is contaminated by the stall it is meant to detect. The reference must be fixed early and the series smoothed before thresholding, which is what the detector of Section 6.5 does. We have not established *why* smoothing is sufficient, only that it is, and the detector remains sensitive to the opening fraction: a run whose first steps are already unproductive gives a poor reference.

What the release cannot support. The natural repair for a judged label is a measured one, and this corpus does not permit it. 79% of the sampled trajectories contain no verifiable success event at all: explicit test output appears in 3% of runs and successful builds in 1%. The one abundant marker, an exit code of zero, is anti-correlated with success — failed runs carry 7.3 of them on average against 3.7 in solved runs — because it records that a command did not raise, not that the task advanced. Two further weaknesses compound this. Redaction (25.5% of observations carrying any content, 29.7% averaged per trajectory) degrades precisely the evidence and verification channels we analyse, and it is the largest single driver of annotator disagreement: fully legible windows tie between readers at 5.3%, partly redacted ones at 23.4. And relevance is computed lexically, so a discovery described in different words than the task scores zero — one reason the evidence channel underperforms, and a limitation of our implementation rather than of the idea behind it.

9 Reproducibility

The pipeline is deterministic and split into stages, each writing a frozen artifact: `build_dataset.py` draws the stratified sample and writes window features; `embed_semantics.py` writes the step representations used by the semantic channel; `make_annotation_cards.py` and its two companions write the annotation cards; `build_gold.py` merges all annotation rounds into `adjudicated.csv`, `agreement.json` and `regions.json`; `run_experiments.py` runs task-level cross-validation and writes `window_features.parquet` and `monitor_scores.parquet` with the metric tables; and `make_figures.py` with `export_results_tex.py` regenerates every figure and every number in this paper from those artifacts. Data are public: Terminal-Bench 2.0 trajectories [4] and SWE-agent trajectories [5]. The monitor itself is about six hundred lines of dependency-free Python (standard library plus NumPy) and runs in milliseconds per step on one CPU core: on 2016 windows it takes 0.33 ms to compute a window’s feature vector and 0.003 ms to score it and evaluate the alarm rule, so monitoring a thousand-step run costs under a second. The only external service used anywhere in the pipeline is the annotation step. A test suite (`run_tests.py`) asserts the invariants the protocol depends on, in particular that truncating a trajectory leaves the features of the surviving windows unchanged, which is the online restriction, and a number audit (`audit_numbers.py`) re-derives the headline quantities straight from the frozen artifacts and compares them with the macros the paper prints; at the time of writing all 42 checked values matched. The figures are checked the same way, by `audit_figure_text.py`, which measures the bounding box of every drawn label and fails on any label-on-label overlap, any label printed outside its own panel, any glyph cut at the raster border, and any label that falls below five points once the figure is scaled to the text width.

10 Conclusion

We asked which observable signals let an online monitor recognise that a coding agent has stopped making progress without knowing the correct solution. The answer our data supports is narrower and more useful than “detect loops”. Exact repetition is a precise but partial detector; semantic redundancy of successive steps is a good ranking signal but blind to direction; workspace churn is close to worthless; verification deltas are rare but nearly decisive; and the signal that adds the most is the cheapest to compute — whether the material the agent is currently uncovering is new *and* about the task. That finding is the one we would want a runtime builder to take away, together with the negative results: the signals that look most natural are not the ones that work, and a monitor that stops healthy runs is worse than no monitor at all.

11 Author contributions and use of AI tools

Both authors contributed to the conception of the study, the design of the codebook, the annotation protocol, the interpretation of the results and the writing of this report. The implementation, the annotation pipeline and the first drafts were produced with an autonomous AI coding agent; its role, the stages it touched, the tools and versions involved and the errors it made are recorded in full in the accompanying AI-assistance log, submitted as supporting material together with the annotation cards, the frozen feature matrices and the monitor implementation. No result reported here was taken on trust from that log: every number in the paper is regenerated from the frozen artifacts by the released scripts (Section 9), and the label set remains open to independent human re-reading.

A Budget-calibrated operating points

The main text reports the raw threshold sweep, in which every monitor uses the same score threshold. That view favours monitors whose scores happen to sit on a convenient scale, so we also calibrated each monitor’s threshold on the training trajectories: for a firing budget b , the threshold is the $(1 - b)$ quantile of that monitor’s training step scores, so every monitor alarms at a comparable rate before the persistence rule is applied. Under this calibration the ordering changes and detection collapses for most monitors: at the 5% budget the hand-designed evidence–semantic monitor catches 0.213 of annotated stagnant windows with 0.014 false stops, and the best learned semantic mixture reaches 0.245 with 0.010. The reason is informative rather than disappointing: the extreme scores inside a run are frequently *productive* moments — the one decisive discovery, the test that finally passes — so a monitor tuned to alarm only at its own extremes is tuned to alarm in the wrong place. We therefore report both views and treat the raw sweep as an optimistic bound.

B Annotation codebook

The codebook given to annotators is reproduced verbatim in the released artifact (`docs/annotation_guide.md`). Its core is the three-channel progress test (§3): a window is productive if new task-relevant information became known, or a plausible artefact changed durably, or verification state improved; it is stagnant if none of those happened while the agent kept acting. Six labels are available, of which two (`BLOCKED_EXTERNAL`, `UNCERTAIN`) are held out of the binary task.

C Window-size sensitivity

The window length w is the one hyperparameter the feature set has, so we re-ran the whole protocol at $w = 6, 20$ and 30 . The window matters, and it is not monotone: at $w = 6$ the monitors have too little context and the semantic family falls to 0.565, with per-window detection at a 5% budget about a fifth of its $w = 10$ value; at $w = 20$ the global ranking is similar (semantic reaches 0.759) but the within-run numbers move substantially; and at $w = 30$ the window is long enough that productive work inside it masks earlier stagnation, costing the evidence channel heavily. The $w = 10$ point used throughout is thus a compromise between context and resolution rather than an optimum we would defend on principle, and a deployment should tune it.

D Cross-scaffold transfer

Does a monitor fitted on some agent scaffolds work on one it has never seen? We trained the same feature sets on annotated windows from every scaffold except one and evaluated on the held-out scaffold, for each of the nine scaffolds in the sample. The ordering from the main experiments survives, but the numbers are lower and the spread is wide: the semantic family averages 0.692 (range 0.450–0.880), against 0.775 trained and evaluated inside the corpus, and every family is close to chance on at least one scaffold. Repetition (0.558) and workspace (0.645) transfer about as well as each other, and task-grounded evidence alone transfers worst (0.564), again improving only in combination with the semantic channel (0.701). These signals are portable

in kind but not in calibration: a deployment should re-fit its threshold per scaffold rather than carry one across agents. A genuine cross-corpus check — the same extractor applied to SWE-agent trajectories over SWE-bench — was prepared but could not be evaluated here, because that corpus carries no stagnation annotations, and reporting alarm rates there without labels would be a claim we cannot support. Adding that annotation set is the first item of future work.

E Channel behaviour, per trajectory

Figure 5 in Section 6 shows the three channels step by step for three annotated trajectories, each containing both labels, so the shaded bands show where the same run stopped progressing.

References

- [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [2] Simarjot Khanna. Zombie agents: Detecting semantic livelock in long-horizon autonomous software. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware)*, 2026. Main track. Reports semantic livelock in 25% of analysed long-duration SWE-agent failures and a 208-step checkerboard oscillation; OpenReview B1VgsZzg0N.
- [3] James Kwok et al. LLM-as-a-Verifier: A general-purpose verification framework. arXiv preprint arXiv:2607.05391, 2026.
- [4] Yoonho Lee. Terminal-Bench 2.0 trajectories. Hugging Face dataset `yooholee/terminalbench-trajectories`, 2026. Full agent trajectories from the Terminal-Bench 2.0 leaderboard; accessed 2026-09-10.
- [5] Nebius. SWE-agent trajectories. Hugging Face dataset `nebius/SWE-agent-trajectories`, 2024. 80,036 SWE-agent trajectories over SWE-bench and SWE-bench-extra; accessed 2026-09-10.
- [6] Opencode. Agent has no effective no-progress/loop detection during repeated file investigation. Issue #43603, anomalyco/opencode, 2026.
- [7] Opencode. Subagent: infinite loop of identical tool calls for ~50 min with no loop protection. Issue #45442, anomalyco/opencode, 2026.
- [8] Dzung Pham, Kleomenis Katevas, et al. AgentStop: Terminating local AI agents early to save energy in consumer devices. arXiv preprint arXiv:2605.15206, 2026.
- [9] Rui Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv preprint arXiv:2607.06184, 2026.
- [10] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks. arXiv preprint arXiv:2608.03222, 2026.
- [11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- [12] Huan Zhang, Mingju Chen, Dongxu Zhou, Can Lv, Heng Chang, Sen Cui, Faguo Wu, and Shiji Zhou. TRCA: Transition-wise rubric credit assignment for long-horizon LLM agents. arXiv preprint arXiv:2608.16156, 2026.
- [13] Xu Zhao et al. Failure as a process: An anatomy of CLI coding agent trajectories. arXiv preprint arXiv:2607.09510, 2026.